HOOK BACKEND · DELIVERY · 31 OCTOBER 2026

Project delivery

The Hook backend has been moved to its own server, made production-ready and tested at high traffic. All work is complete, merged into the main branch and live.

Delivered

ITEM	DESCRIPTION
Server	Dedicated server (Hostinger KVM 4) running the backend
Docker	Every part of the backend runs in its own container
Load balancing	Nginx spreads traffic across 3 copies of the API
Cloudflare	HTTPS, attack protection, caching and DNS
Firewall	Only Cloudflare can reach the server
Event-driven system	RabbitMQ handles payments, orders and fulfilment separately
Monitoring	Grafana dashboards, logs, uptime checks and email alerts
Automatic deployment	Changes merged on GitHub are tested and deployed automatically
Modular codebase	Code organised into 8 business modules
Prod and dev environments	Two separate environments on the same server
Load testing	Tested with up to 30,000 shoppers at the same time

Environments

	ADDRESS	DATABASE
Production	api.hookapp.tech	New production cluster
Development	dev-api.hookapp.tech	Development cluster

Test results

TEST	RESULT
10,000 shoppers	99.6% of requests succeeded
30,000 shoppers in 20 seconds	98.7% succeeded, no server errors

Deliverables and access

Deliverables

DELIVERABLE	WHERE
Source code, all work merged	GitHub: `Hook-Deliveries/Hook-Backend`, branch `main`
Production environment	`https://api.hookapp.tech`
Development environment	`https://dev-api.hookapp.tech`
Configured server	Hostinger KVM 4, `153.92.222.181`
Automatic test and deploy pipeline	GitHub → Actions tab
Cloudflare setup (DNS, HTTPS, firewall, caching)	Cloudflare → `hookapp.tech`
Monitoring dashboards and email alerts	See the table on the right

What the Hook team needs for access

	ACCESS	HOW
1	Server access	Use the Windows SSH steps on page 3. Use the deploy account for project work and dashboard tunnels.
2	Dashboard logins	Grafana, Uptime Kuma and RabbitMQ logins, shared privately
3	GitHub	Write access to `Hook-Deliveries/Hook-Backend` to see tests and deploys
4	Cloudflare	Access to the `hookapp.tech` account for traffic and DNS
5	Alert emails	Sent to hook.africa2025@gmail.com; more addresses can be added

Dashboards to track the app

DASHBOARD	WHAT IT SHOWS	OPEN AT
Grafana	Traffic, errors, response times, server CPU, memory and disk, event queues and logs. Three dashboards: Hook Overview, Hook Application and VPS Server	`localhost:3000`
Uptime Kuma	Whether the API is up, response time and uptime history	`localhost:3001`
RabbitMQ (prod)	Payment, order and fulfilment event queues	`localhost:15673`
RabbitMQ (dev)	Development event queues	`localhost:15674`
Cloudflare Analytics	Visitors, requests, cache hits and blocked attacks	dash.cloudflare.com
GitHub Actions	Every test run and deployment	github.com

The dashboards are private. In a second Windows Command Prompt, open this tunnel, then visit the addresses above:

```
ssh -N -i "%USERPROFILE%\.ssh\id_ed25519" ^
  -L 3000:127.0.0.1:3000 ^
  -L 3001:127.0.0.1:3001 ^
  -L 15673:127.0.0.1:15672 ^
  -L 15674:127.0.0.1:15674 ^
  root@153.92.222.181
```

Keep the window open while you use the dashboards. Alerts arrive by email without opening anything.

Delivered by: Ogunseitan Gold Enoch · Date: 31 October 2026

Received for Hook by: _____ · Date: _____

HOOK BACKEND · OPERATIONS · NOVEMBER 2026

SSH access and environment updates

Use this runbook from the operator's Windows PC. Keep secrets in the server environment files only.

1. Connect from Windows Command Prompt

Open Command Prompt on the PC that holds her private key and run:

```
ssh -o IdentitiesOnly=yes -i "%USERPROFILE%\.ssh\id_ed25519" root@153.92.222.181
```

At the passphrase prompt, enter the key's passphrase. It is not the VPS password. If SSH asks to trust a host key, verify its fingerprint in Hostinger before accepting it.

2. Switch to the deployment account

```
sudo -iu deploy
whoami
```

The result should be `deploy`. Run project commands only after SSH connects and the remote Linux prompt appears.

3. Open the existing main environment file

```
cd /home/deploy/hook-backend/deploy
if [ -f .env ]; then nano .env; else echo ".env missing - stop"; fi
```

This checks that the real file exists before opening Nano. If it reports missing, do not create an empty file; confirm the project path first.

Values to set in both environments

Edit the existing lines; do not add duplicate names. Enter the API key and secret directly in Nano, with screen sharing paused.

```
CLOUDINARY_CLOUD_NAME=df4f0usnh
CLOUDINARY_API_KEY=(enter privately)
CLOUDINARY_API_SECRET=(enter privately)
CLOUDINARY_UPLOADS_ENABLED=true
CLOUDINARY_UPLOAD_FOLDER=hook
CLOUDINARY_UPLOAD_PRESET=ProfileX
PAYMENT_WEB_ORIGIN=https://app.hook.africa
PUBLIC_WEB_URL=https://app.hook.africa
```

Save with Ctrl+O, Enter, then Ctrl+X. Check the variables without printing secrets, then apply the main change:

```
chmod 600 .env
docker compose up -d --force-recreate api-1 api-2 api-3 worker consumer-payments
consumer-orders consumer-fulfilment
docker compose ps
```

Container recreation applies the new values and can briefly interrupt requests. `PAYMENT_WEB_ORIGIN` is used for hosted payment links. `PUBLIC_WEB_URL` is documented here but is not currently read by backend code.

Development is a separate checkout. Its expected file is `/home/deploy/hook-dev/deploy/.env`. Verify the directory and existing file before editing. Do not create it if missing.

Payment provider endpoints. Main callbacks: Paystack `https://api.hookapp.tech/api/v1/payments/paystack/callback`; Monnify `https://api.hookapp.tech/api/v1/payments/monnify/callback`. Register webhooks separately: `https://api.hookapp.tech/api/v1/webhooks/paystack` and `https://api.hookapp.tech/api/v1/webhooks/monnify`. Development uses `https://dev-api.hookapp.tech` with the same callback and webhook paths. Both environments currently use Paystack test credentials and Monnify sandbox mode; replace main credentials with live keys before accepting real payments. The app return URI is `hook://payments/return`.

Cloudinary checks for main and development

What the code expects

Catalog media uploads are signed and require `CLOUDINARY_CLOUD_NAME`, `CLOUDINARY_API_KEY`, and `CLOUDINARY_API_SECRET`. They do not use the upload preset. The general upload route uses signed uploads when the API key and secret are present; it uses the unsigned preset only when those credentials are absent.

`CLOUDINARY_UPLOADS_ENABLED` gates production startup validation.
`CATALOG_SIGNED_MEDIA_ENABLED` controls the signed catalog upload route.

Firewall cause and verified fix · 10 November 2026

The Cloudflare-only Docker filter was also dropping established replies for outbound container HTTPS, including Cloudinary. The drop counter rose during a failed API request. The firewall script now passes established IPv4 and IPv6 replies before Cloudflare filtering; new inbound requests still require Cloudflare.

After applying the fix, signed Cloudinary API pings and image fetches returned HTTP 200 in both stacks. A temporary signed image upload passed in main and development, and both test images were deleted. Both public API health checks return HTTP 200.

Safe check from each deploy folder

```
docker compose ps
docker compose exec -T api-1 node -e 'for (const n of
["CLOUDINARY_CLOUD_NAME","CLOUDINARY_API_KEY","CLOUDINARY_API_SECRET"])
console.log(n+": "+(process.env[n]?"set":"missing"))'


docker compose exec -T api-1 node -e 'const h=require("https");const
u="https://api.cloudinary.com/v1_1/"+process.env.CLOUDINARY_CLOUD_NAME+"/ping";const
q=h.get(u,
{auth:process.env.CLOUDINARY_API_KEY+":"+process.env.CLOUDINARY_API_SECRET,timeout:7
000},r=>{console.log("Cloudinary HTTP "+r.statusCode);r.resume()});q.on("timeout",
()=>q.destroy(new Error("timeout")));q.on("error",e=>console.log("Cloudinary request
failed: "+(e.code||e.message)))'
```

Run the checks separately in main and development. They report presence/status only; never print the env file.

Before touching development

Check that `/home/deploy/hook-dev/deploy/.env` exists. Inspect only the non-secret isolation settings:

```
cd /home/deploy/hook-dev/deploy
grep -E '^(COMPOSE_PROJECT_NAME|API_IMAGE|API_PORT|API_BIND|SKIP_NGINX)=' .env
```

The Compose defaults use project `hook`, image `hook-api`, and Nginx ports 80 and 443. If both stacks are running, development must use an independent Compose project and image, a distinct API port, and must not start a second Nginx on the same ports. If unclear, stop and ask the server owner before running Compose.

After isolation is verified, update development

```
if [ -f .env ]; then nano .env && chmod 600 .env; else echo ".env missing - stop";
fi
docker compose up -d --force-recreate api-1 api-2 api-3 worker consumer-payments
consumer-orders consumer-fulfilment
docker compose ps
```

Run this only from the development deploy directory after confirming the project name, image, port, and Nginx setup are separate from main.

GitHub deploy behavior

A successful CI run on `development` deploys development; a successful CI run on `main` deploys production. GitHub does not copy server `.env` files. Update and verify each server environment separately before relying on a push.

> **Keep credentials private.** Never commit API keys or secrets, paste env contents into chat, or share a screen while editing them. Rotate credentials that have been exposed, then update both server files and recreate their app containers.